

Lab Assessment Rubrics of Digital System Design (CPE-344)										
Lab Number			09							
Lab Title			Implementation of multipliers and designing of ALU							
Submitted by										
Names			Registration Number							
Sumaira Safeer			FA22-BCE-019							
Rubrics name & number								Marks		
								In-Lab	Post-Lab	
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.									
	R3: Interpretation of Subject Knowledge: Ability to interpret and explain mathematical and/or visual forms, including equations, diagrams, graphics, figures, and tables.									
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.									
Design	R7: Implementing Design Strategy: Ability to execute a solution taking into consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram]									
	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.									
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.									
	R11: Tools Evaluation: Ability to simulate the experiment and them using hardware tools to verify The results.									
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.									
	R13: Management of Teamwork: Ability to appreciate, understand and work with multidisciplinary team members.									
Rubrics to follow										
Rubrics	R2	R3	R5	R6	R7	R8	R9	R11	R12	R13
In-Lab										
Post-Lab										

Lab: 09

1. Objectives:

Objective of this lab is to get familiar with implementation of

- Various sequential circuits
- N-bit counter with asynchronous
- N-bit behavioral Adder
- 4-bit behavioral Adder
- N-bit comparator using relational operator

Lab Tasks:

1.1. In Lab Task#01:

Implement N bit counter with asynchronous

Solution:

- **Verilog Code**

```
module counterNBitWithAsynchronous #(parameter N = 4)(input wire clr , input wire clk ,
output reg [N-1:0] q);
always @(posedge clk or posedge clr)
begin
if(clr == 1)
q <= 0;
else
q <= q + 1;
end
endmodule
```

- **Verilog Test Bench**

```
module counterNBitWithAsynchronousTestBench;
reg clr;
reg clk;
wire [3:0] q;
counterNBitWithAsynchronous uut (
.clr(clr),
.clk(clk),
.q(q)
);
initial begin
clr = 1;
clk = 1;
#100;
clr = 0;
clk = 0;
#100;
```

end
endmodule

1.2. In Lab Task#02:

Implement N bit behavioral adder

Solution:

- Verilog Code

```
module nbitAdder #(parameter N = 8)(input wire [N-1:0] a,  
input wire [N-1:0] b,  
output reg [N-1:0] y);  
always @(*)  
begin  
y = a + b;  
end  
endmodule
```

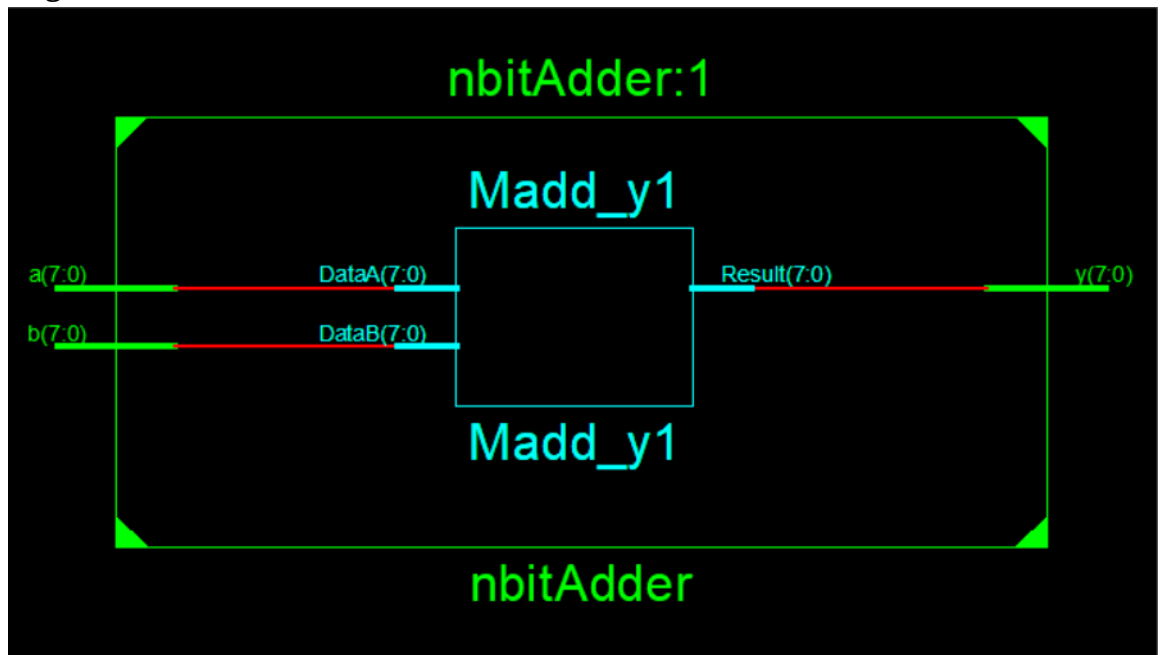
- Verilog Test Bench

```
module nbitAdderTestBench;  
reg [7:0] a;  
reg [7:0] b;  
wire [7:0] y;  
nbitAdder uut (  
    .a(a),  
    .b(b),  
    .y(y)  
);  
initial begin  
    a = 8'b00000000;  
    b = 8'b11111111;  
    #100;  
    a = 8'b00101010;  
    b = 8'b10110101;  
    #100;  
    a = 8'b00111110;  
    b = 8'b10111110;  
    #100;  
    a = 8'b00010100;  
    b = 8'b10100110;  
    #100;  
    a = 8'b11010101;  
    b = 8'b11100110;  
    #100;  
end  
endmodule
```

- Waveform



- RTL Diagram



1.3. Task#03:

Implement 4 bit behavioral adder

Solution:

- Verilog Code

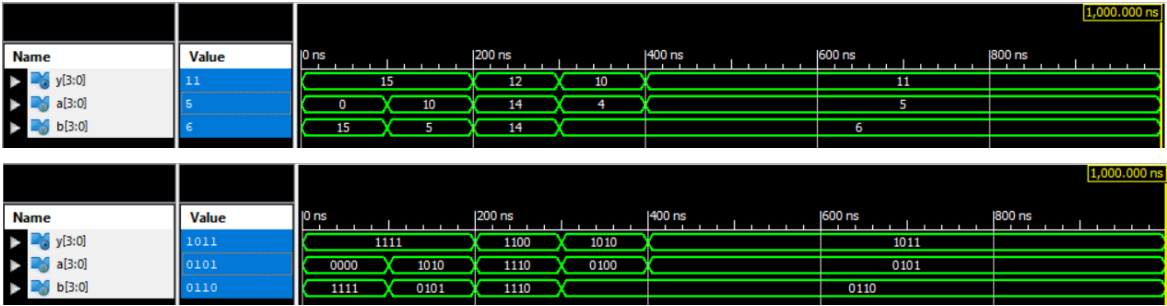
```
module nbitAdder #(parameter N = 4)(input wire [N-1:0] a, input wire [N-1:0] b, output reg [N-1:0] y);
always @(*)
begin
y = a + b;
end
endmodule
```

- Verilog Test Bench

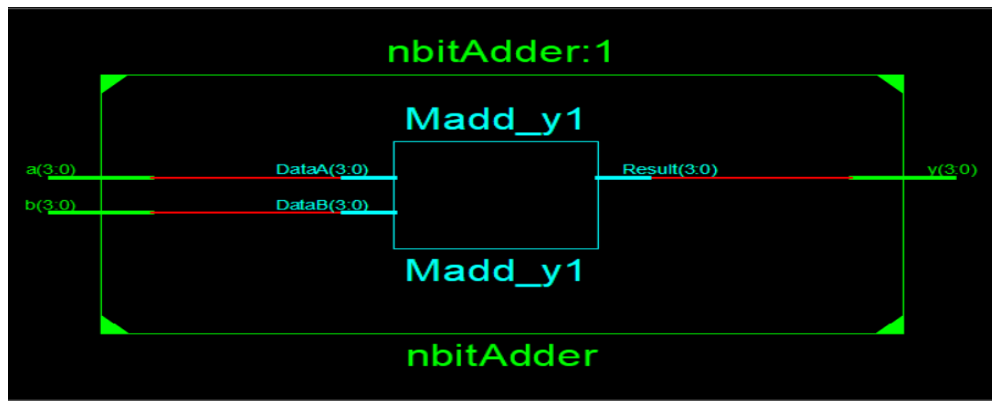
```
module nbitAdderTestBench;
reg [3:0] a;
```

```
reg [3:0] b;
wire [3:0] y;
nbitAdder uut (
    .a(a),
    .b(b),
    .y(y)
);
initial begin
    a = 4'b0000;
    b = 4'b1111;
    #100;
    a = 4'b1010;
    b = 4'b0101;
    #100;
    a = 4'b1110;
    b = 4'b1110;
    #100;
    a = 4'b0100;
    b = 4'b0110;
    #100;
    a = 4'b0101;
    b = 4'b0110;
    #100;
end
endmodule
```

• Waveform



• RTL Diagram



1.4. Post Lab Task#04:

Implement of N bit Comparator using relational operator

Solution:

- **Verilog Code**

```
module comp
#(parameter N = 4)
(input wire [N-1:0] x, input wire [N-1:0] y, output reg gt, output reg eq, output reg lt);
always @(*)
begin
gt = 0;
eq = 0;
lt = 0;
if(x > y)
gt = 1;
if(x == y)
eq = 1;
if(x < y)
lt = 1;
end
endmodule
```

- **Verilog Test Bench**

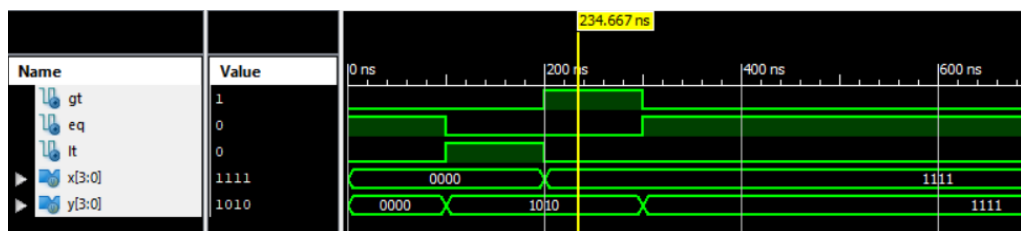
```
module compTest;
reg [3:0] x;
reg [3:0] y;
wire gt;
wire eq;
wire lt;
comp uut (
.x(x),
.y(y),
.gt(gt),
.eq(eq),
```

```

        .lt(lt)
    );
    initial begin
        x = 4'b0000;
        y = 4'b0000;
        #100;
        x = 4'b0000;
        y = 4'b1010;
        #100;
        x = 4'b1111;
        y = 4'b1010;
        #100;
        x = 4'b1111;
        y = 4'b1111;
        #100;
    end
endmodule

```

- Waveform



- RTL Diagram

